

Getting started with Navon Cloud

A guide for university researchers and teams

Navon Cloud gives your researchers serious compute in the browser: GPUs, notebooks, models and tools, with no procurement cycle and nothing to install locally. This guide takes a first-time user from signing in to running their own work, and ends with a thorough FAQ. Keep it as your reference.

What's inside: the core ideas, your first workspace, grids, the apps you can run, connecting from your laptop, running Python / Hugging Face models / ANSYS / MATLAB, data and storage, a short platform reference, and the FAQ.

The core ideas in 60 seconds

- **Project.** A folder that groups related apps together (one experiment, one course, one team).
- **App.** A running container: a notebook, a model, a tool, or your own Docker image.
- **Grid.** A group of servers your apps run on. You get two:
 - **High-Performance grid** for heavy, long jobs (training, multi-day simulation).
 - **Shared grid** for everyday interactive work and the shared AI models.
- **Server / node.** A machine in a grid, each with an **NVIDIA A100 80GB GPU**, 22 vCPUs, 118 GB RAM and 968 GB storage.
- **Endpoint.** The link that opens your running app, shown at the top right of its page.

Your first workspace

1. Sign in to the **Navon portal** (the link is in your onboarding email).
2. Create a **project**.
3. Click **Create App**, open the **App Gallery**, and choose **JupyterLab** (notebooks) or **code-server** (a browser version of VS Code).
4. Choose **Quick Launch**, then pick the **grid** your administrator shared with you.
5. Wait about a minute, then open the app from its **endpoint**. You now have a GPU workspace in your browser.

Each app has tabs to **Logs** (live output), **Terminal** (a command line inside the app), and **Metrics** (performance).

Grids: where your work runs

You do not choose or pay for hardware. Your administrator shares grids with you, and you run apps on them.

- Do your everyday development, notebooks, inference and short jobs on the **Shared grid**. This is also where the shared AI models live.

The apps you can run

Open the **App Gallery** for ready-made apps, including:

- **Notebooks and editors:** JupyterLab, code-server.
- **AI chat and models:** Open WebUI (chat with open models such as Qwen and DeepSeek), and high-throughput serving with vLLM (Llama-3.3-70B, Qwen3 and more).
- **Data and search:** MinIO (large-file storage), Qdrant (vector search), Meilisearch (text search).
- **Media and language:** Whisper (speech to text), LibreTranslate (translation).
- **Dashboards and automation:** Grafana, n8n, Appsmith.
- **Utilities:** Gitea (git), FileBrowser, and more.

The gallery is a shortcut list, not a limit. Because your apps have open internet access, you can also run **any** open-source tool or model, or deploy **your own Docker image**.

Connecting from your laptop (SSH)

You can work in the browser, or connect directly from your own machine over SSH.

Step 1. Make an SSH key. Open Terminal (Mac) or PowerShell (Windows) and run:

```
ssh-keygen -t ed25519
```

Press Enter a few times to accept the defaults. This creates a private key (stays on your laptop) and a public key (safe to share).

Step 2. Register your public key. In the portal, go to **Settings → SSH keys → Add key**, name it, and paste the contents of your public key file (`id_ed25519.pub`). Never paste the private key.

Step 3. Turn on access. On the server, open **Node Actions → SSH access**, tick your key, and Save. A server can have many keys, so a whole team can connect to the same node.

Step 4. Connect. Use the address, username and port shown for the server:

```
ssh USERNAME@SERVER-ADDRESS -p PORT
```

Using an IDE:

- **VS Code:** install the "Remote - SSH" extension, then Connect to Host with the same details.
- **Visual Studio (Windows, for C/C++):** install the "Linux and embedded development with C++" workload, add the connection under **Tools → Options → Cross Platform → Connection Manager**, then target a CMake or Linux C++ project at it.

Running your software

Python (ready today). Launch JupyterLab or code-server, then `pip install` what you need. Confirm the GPU with `nvidia-smi` in a terminal, or `torch.cuda.is_available()` in Python. For long jobs, run from the Terminal and use a High-Performance slot.

Any Hugging Face model, including private ones. Create a Hugging Face **Read** token (huggingface.co,

```
pip install -U transformers huggingface_hub torch accelerate
huggingface-cli login          # paste your token
```

Then load any model, public or private:

```
from transformers import pipeline
asr = pipeline("automatic-speech-recognition",
               model="your-username/your-private-model", token=True, device=0)
```

Set the token as an `HF_TOKEN` environment variable in the app's Runtime step so it never sits in your code.

ANSYS. Build a Docker image with ANSYS installed and deploy it as a custom app with the GPU attached (use Docker Link for large images). Point it at your university's licence server with environment variables, then run it headless or drive it from Python with PyMAPDL.

MATLAB. Deploy the official `mathworks/matlab` image with the GPU attached. Expose its web port to get MATLAB in the browser, and license it with your network licence manager or an online sign-in.

Licences: for commercial software such as ANSYS and MATLAB, you bring your own licence. Navon supplies the compute, and we help you wire the licence up.

Data and persistence

- Keep large datasets and results in **MinIO** (object storage), or on storage mounted to your app.
- Anything built into a **Docker image** always returns when the app starts, on any server.
- Files you write while an app is running survive a simple stop and start. They are lost on a settings change, on deletion, or if the app moves to another server.
- To keep local files, set the app to **Stateful** (Config, Execution), bake what you need into the image, or use a Post-start command. For anything important, store it in MinIO.

Platform reference (quick)

When you create a custom app there are five steps:

- **Source.** Where the app comes from: search DockerHub, use Docker Link for a private registry, connect a GitHub repo, or upload an image (up to 30 GB). Use Docker Link for large images.
- **Runtime.** Environment variables, restart policy, health checks.
- **Network.** Expose a port and choose a type: HTTPS Load Balancer for web apps, TCP Load Balancer for SSH or remote desktop. Use the Inbound IP Allowlist to restrict access to your campus. For a TCP Load Balancer the external port is assigned for you, so read the exact address and port from the Endpoints panel.
- **Hardware.** The minimum CPU, GPU and memory your app needs.
- **Location.** The grid to run on. Pick a grid with A100s to use the GPU.

Other useful facts:

- Outbound internet is open, so licence servers, package repositories and model hubs are reachable

- There is **no private grid-to-grid network**. Multi-node jobs communicate over normal TCP/UDP on the internet using each app's address.

Getting help

- For access, grids, and account questions: your university administrator or your Navon contact.
 - For platform how-to and technical issues: Navon support.
-

Frequently asked questions

Getting started

What is Navon Cloud? A managed cloud that gives you GPUs, notebooks, AI models and tools in the browser. You run apps on servers grouped into grids, with nothing to install locally.

Do I need to install anything? No. Everything runs in the browser. You only install an SSH client if you want to connect from your own laptop, and most laptops already have one.

How do I get an account? Your university administrator or Navon contact sets you up and shares a grid with your account. Your onboarding email has the portal link.

What is the difference between a project, an app and a grid? A project groups related apps. An app is one running tool or model. A grid is the set of servers your apps run on.

Grids and GPUs

What GPU do I get? Each server carries an NVIDIA A100 80GB GPU, with 22 vCPUs, 118 GB RAM and 968 GB storage.

High-Performance or Shared grid, which should I use? Use Shared for everyday interactive work and short jobs. Use High-Performance for heavy or long-running jobs such as training and multi-day simulations, and ask your administrator to book a slot.

How do I make sure my app uses the GPU? Pick a grid that contains A100s when you create the app. Then confirm with `nvidia-smi` or `torch.cuda.is_available()`.

Can several people share one grid or server? Yes. A grid can serve many researchers, and a single server can authorise many SSH keys, so a team can share it.

Apps, software and models

The tool or model I need is not in the App Gallery. The gallery is only a shortcut list. Because your apps have open internet access, you can install any open-source tool, or deploy your own Docker image.

Can I run my own Docker image? Yes. Search DockerHub, connect a private registry with Docker Link, connect a GitHub repo, or upload an image up to 30 GB.

How do I load a model from Hugging Face? Launch JupyterLab, install `transformers`, and load the model by its repository id. For a private model, log in first with a Hugging Face Read token.

Can I use my own private, fine-tuned model? Yes. Log in with your Hugging Face token, and the platform can pull your private repository.

How do I let others chat with an AI model? Launch Open WebUI or a vLLM model from the gallery and share its endpoint.

Do you provide ANSYS or MATLAB licences? No. You bring your own licence for commercial software. Navon supplies the compute, and we help you connect the licence.

Connecting

Can I use VS Code or Visual Studio? Yes. VS Code connects with the "Remote - SSH" extension. Visual Studio connects for C/C++ work through its Connection Manager. Both use the same SSH details.

Can I use JupyterLab or RStudio? JupyterLab and code-server are one click in the gallery. For R, install it inside a JupyterLab or code-server session, or ask us to set it up.

Can my whole team connect to the same server? Yes. Register each person's public key and authorise the ones you want on that server.

Data, storage and networking

Where should I keep my data? In MinIO (object storage) or on storage mounted to your app, especially anything you want to keep long term.

Will my files survive a restart? Files survive a simple stop and start. They are lost on a settings change, deletion, or a server move. Keep important data in MinIO, set the app to Stateful, or build it into the image.

Can my app reach the internet? Yes. Outbound access is open, so package repositories, model hubs and licence servers are all reachable.

How do I expose a web app or a port? In the Network step, add the port and choose HTTPS Load Balancer for a web app or TCP Load Balancer for SSH or remote desktop, then read the address and port from the Endpoints panel. Use the Inbound IP Allowlist to limit access to your campus.

Can I run a job across several servers? Yes, but there is no private grid network, so the parts communicate over normal internet TCP/UDP using each app's address. Tell us early if you need large multi-node runs so we can help set them up.

Security

Is my work private? Each app is isolated, access is controlled by your administrators, and your data and models stay yours.

Is it safe to share my SSH key? Yes, if it is the public key. Never share the private key, and never paste it into any website.

Troubleshooting

My app is slow to start. Provisioning takes about a minute. Very large images take longer, and public image registries can rate-limit downloads when many apps start at once.

I lost my files after changing a setting. Changing an app's configuration recreates the container, which clears local files. Keep data in MinIO, set the app to Stateful, or build it into the image.

ssh-keygen is not found on Windows. Add the OpenSSH Client: Settings, Apps, Optional Features, Add "OpenSSH Client", then try again.

My grid is not showing in my account. That is an access question, not something you did wrong. Contact your administrator to share or confirm the grid.